

The Claw Workforce Engine

Full Dual-System Technical Sheet & Subsystem Architecture

Hybrid Public-Private Model

Zero Manual Applications

Algorithmic Claw Agent

Version 1.0

Executive Architecture Summary

The Claw Workforce Engine is a closed-loop, hardware-anchored hiring ecosystem operating on a hybrid public-private partnership model. By eliminating traditional manual job applications, job portals, and drawn-out interview scheduling, the system combines municipal kiosk infrastructure with enterprise-embedded terminals. Candidates interact once with an automated kiosk, after which the proprietary *Claw Private Agent* manages resume tailoring, cover letter generation, background check verification, and automated daily weighted lottery matching.

1. High-Level Architecture & Ecosystem Flow

The system connects municipal jobseekers directly with enterprise labor demands via a closed-loop selection pipeline:

Applicant → **Kiosk Intake** → **Claw Private Agent** → **Workforce Network** → **Employer Portal** → **Daily Weighted Lottery** → **Daily Job Assignment**. No manual applications are ever submitted by jobseekers.

Layer / Subsystem	Components & Key Responsibilities
Client & Interface Layer	Kiosk UI: Touchscreen intake, biometric ID scan, microphone/camera WebRTC video recorder. Employer Web Portal: Job creation, candidate ranking, interview viewer, lottery rules. Admin Console: Public workforce operators & enterprise governance, bias monitoring.
Service Layer (Backend)	Microservices: Identity & Auth, Applicant Profile, Document & Media, Claw Agent Orchestration, Matching & Scoring Engine, Lottery Engine, Notification Service (SMS/IVR), Employer Job Service, Interview & Scheduling, Audit & Compliance Service.
Data Layer	Relational Core DB: Core operational entities (Applicants, Jobs, Employers, Snapshots, Assignments). Object Storage: Resumes, cover letters, WebRTC universal video interview files. Search Index & Feature Store: Full-text skills matching index and ML features for probability weights.

2. UI/UX Wireframe Specifications

Interface Module	Wireframe Component & Interaction Specifications
Kiosk Intake UI (Touchscreen Terminal)	• Step 1 (Intake Form): Phone OTP verification, Government ID document scanner, structured form for availability, transport radius, skill categories, and optional work barriers. • Step 2 (Document Ingestion): Base resume upload & background check consent. • Step 2.2 (Video Interview): Touchscreen WebRTC video recorder capturing narrative introduction, general interview, and skill/scenario demonstrations.
Employer Web Portal (Enterprise Dashboard)	• Job Creation Dashboard: Entry of job title, pay range, schedule, location, and required skills. • Candidate Slate Viewer: Ranked candidate lists with direct access to Universal Interview Video playback. • Lottery Rule Configuration: Controls to adjust scoring weight parameters (skills, reliability, interview) and assignment confirmation.

Interface Module	Wireframe Component & Interaction Specifications
Admin & Governance Console (Municipal/Enterprise)	• Operator Dashboard: Governance interface for public workforce agencies and corporate admins. • Bias & Equity Metrics: Real-time monitoring of disparate impact metrics across demographic and geographic vectors.

3. System API & Integration Specification

Endpoint / Path	HTTP	Description & Operational Payload
/api/v1/lottery/draw	POST	Triggers automated daily selection lottery for a given job ID based on candidate pool snapshots and employer weighting rules.
/api/v1/lottery/draws/{draw_id}	GET	Retrieves complete immutable audit logs, random seed references, fairness metrics, and selected candidate ID arrays for municipal compliance.
/jobs/{job_id}/candidates	GET	Fetches candidate snapshot pool ranked by composite match scores and lottery eligibility status.
/jobs/{job_id}/rules	POST	Updates employer-specific lottery rule parameters (weighting coefficients and public-private equity boost settings).
/assignments/{assignment_id}/decision	POST	Submits employer accept/decline decision or status confirmation for dispatched candidate assignments.

Security & Authentication Protocols: Enterprise employers and system administrators authenticate via **OAuth2 / JWT tokens**. Kiosk hardware endpoints utilize **device-level hardware tokens** for secure communication. All Personally Identifiable Information (PII) is encrypted at rest, isolated from the ML feature store, and enforced via strict **Role-Based Access Control (RBAC)**.

4. Probability Algorithm Deep Dive (Workforce Lottery Engine)

The core daily mechanism selects workers based on weighted probability rather than arbitrary queueing or manual screening. For each applicant *i* in an active candidate pool for a job, the system calculates a composite probability weight **W_i**:

Mathematical Model & Rules

Composite Probability Weight Formula:

$$W_i = (MatchScore \times w_{skills}) + (Reliability \times w_{reliability}) + (InterviewScore \times w_{interview}) + (Background \times w_{background})$$

Public-Private Equity Multiplier: If priority_status == True: $W_i = W_i \times (1.15 \text{ to } 1.20)$

Probability Normalization: $P(\text{applicant}_i) = W_i / \sum W_j$ | **Weight Floor:** $W_i = \max(W_i, 0.001)$

Algorithmic Property	Technical Implementation & Audit Safeguards
Weighted Sampling	Uses weighted random sampling without replacement (np.random.choice(..., replace=False, p=probabilities)) to select applicants for available openings.
Cryptographic Audit Logging	Every daily lottery execution logs the pseudo-random seed used alongside input feature vectors, creating an immutable audit trail for municipal regulatory verification.
Disparate Impact Safeguards	The Audit & Compliance microservice continuously evaluates selection ratios across demographic and geographic vectors to prevent systemic algorithmic bias.

5. Claw Agent & Lottery Engine Real-Code Implementation

Below is the production-grade Python implementation of the **Claw Private Agent Orchestration Service** and the **Workforce Lottery Engine**, featuring dynamic resume tailoring, automated cover letter generation, background check integration, application snapshot generation, and weighted probability selection.

Part A: Data Schemas & Models

```
from datetime import datetime
from typing import List, Dict, Optional
import numpy as np
from pydantic import BaseModel, Field

# --- Core Pydantic Schemas ---
class ApplicantProfile(BaseModel, extra="allow"):
    applicant_id: str
    skills: List[str]
    reliability_signal: float = Field(..., ge=0.0, le=1.0)
    kiosk_interview_score: float = Field(..., ge=0.0, le=100.0)
    background_clearance: bool
    priority_status: bool = False # Public-Private Equity Boost flag

class JobOpening(BaseModel):
    job_id: str
    employer_id: str
    title: str
    requirements: List[str]
    openings_count: int
    weight_skills: float = 0.4
    weight_reliability: float = 0.3
    weight_interview: float = 0.2
    weight_background: float = 0.1
    equity_boost_multiplier: float = 1.15

class ApplicationSnapshot(BaseModel):
    snapshot_id: str
    applicant_id: str
    job_id: str
    match_score: float
    status: str = "active_pool"
```

Part B: Claw Agent Orchestration Service

```
# --- Claw Private Agent Orchestration Service ---
class ResumeRevisionEngine:
    @staticmethod
    def tailor_resume(base_resume: str, job_description: str, matched_skills: List[str]) -> str:
        return f"=== TAILORED RESUME ===\nMatched Skills: {'', ' '.join(matched_skills)}"

class CoverLetterGenerator:
    @staticmethod
    def generate_cover_letter(applicant_name: str, job_title: str, employer_name: str, skills: List[str]) -> str:
        return f"Dear {employer_name},\nAutomatically matched for {job_title} via Claw Agent."

class AutoApplicationEngine:
    @staticmethod
    def process_automatic_application(applicant: ApplicantProfile, job: JobOpening, base_resume: str) -> ApplicationSnapshot:
        matched_skills = list(set(applicant.skills).intersection(set(job.requirements)))
        skill_ratio = len(matched_skills) / max(len(job.requirements), 1)

        match_score = (
            (skill_ratio * 100) * job.weight_skills +
            (applicant.reliability_signal * 100) * job.weight_reliability +
            applicant.kiosk_interview_score * job.weight_interview +
            (100.0 if applicant.background_clearance else 0.0) * job.weight_background
        )
        return ApplicationSnapshot(
            snapshot_id=f"snap_{applicant.applicant_id}_{job.job_id}",
            applicant_id=applicant.applicant_id,
            job_id=job.job_id,
            match_score=round(match_score, 2),
            status="active_pool"
        )
```

Part C: Weighted Workforce Lottery Selection Engine

```
# --- Weighted Workforce Lottery Engine ---
class WorkforceLotteryEngine:
    @staticmethod
    def execute_daily_draw(job: JobOpening, candidate_pool: List[ApplicationSnapshot], profiles_lookup: dict) -> dict:
        if not candidate_pool:
            return {"status": "failed", "reason": "Empty pool"}

        weights = []
        for snapshot in candidate_pool:
            profile = profiles_lookup[snapshot.applicant_id]
            weight = snapshot.match_score
            if profile.priority_status:
                weight *= job.equity_boost_multiplier
            weights.append(max(weight, 0.001))

        total_weight = sum(weights)
        probabilities = [w / total_weight for w in weights]

        selection_size = min(job.openings_count, len(candidate_pool))
        selected_indices = np.random.choice(len(candidate_pool), size=selection_size, replace=False, p=probabilities)
        selected_snapshots = [candidate_pool[i] for i in selected_indices]

        return {
            "draw_id": f"draw_{job.job_id}_{datetime.utcnow().strftime('%Y%m%d')}",
            "job_id": job.job_id,
            "selected_applicants": [s.applicant_id for s in selected_snapshots],
            "random_seed_used": np.random.get_state()[1][0]
        }
```

6. Hardware Engineering & Infrastructure Sheet

Hardware Subsystem	Component Specifications & Operational Role
Touchscreen Display Unit	Industrial-grade ruggedized touchscreen interface for candidate intake forms, skill selections, and preference configuration.
Biometric & Document Scanner	High-resolution optical ID scanner capturing driver's licenses, state IDs, phone OTP input, and automated background check consent.
Camera & Audio Array	HD camera and noise-canceling microphone array supporting WebRTC streaming for Universal Video Interview recordings directly to Object Storage.
Local Computing Unit	Embedded secure computing unit running local kiosk intake application, hardware token authentication, and secure encrypted transit to microservices.
Dual Infrastructure Model	• Municipal Deployment: Installed across public workforce centers, unemployment offices, transit hubs, community centers, shelters, and libraries. • Enterprise Deployment: Embedded terminals situated directly inside logistics hubs, retail centers, and healthcare facilities.

7. Daily Assignment Engine & Post-Lottery Dispatch

Operating immediately after the Workforce Lottery concludes each morning, the Daily Assignment Engine automates frictionless dispatch and state management for selected candidates.

Operational Subsystem	Workflow Mechanism & Integration Details
Automated Multi-Channel Dispatch	• SMS Outreach: Instant SMS containing work date, shift time, location, granular job instructions, and supervisor contacts. • Interactive Voice Response (IVR): Outbound TTS phone calls confirming candidate availability in real time.
Fallback & Micro-Lottery Protocol	Imposes a strict 30-minute confirmation window . If an applicant fails to confirm, the engine automatically releases the slot and triggers an instant micro-lottery re-draw from the active pool.
HRMS & Payroll Sync	Pushes confirmed assignment records directly to enterprise HRMS and payroll software to automatically initialize onboarding documentation.